🩸 ROTA VITAL — BANCO DE SANGUE INTELIGENTE

✅ 100% HOMOLOGADO

Dossiê Executivo de Entrega — Unidade 2 — Desafios PI (Parte 1)

Disciplina: Infraestrutura de Software (Sistemas Operacionais) & AED | **Semestre:** 2026.2 | **Prazo:** 20/10/2026

1. Identificação da Equipe e Responsabilidades

Integrante	Função	Responsabilidades Principais
Eduardo Borges	Desenvolvedor Backend	Camada de persistência relacional, JDBC puro, migrações Flyway e adaptadores de armazenamento.
Luiz Henrique Rocha	Tech Leader	Arquitetura de portas/adaptadores, isolamento de perfis Spring (`u1` vs `infra`) e governança técnica.
Eliziane Mota	QA Engineer	Condução dos ciclos TDD (Red → Green → Refactor), critérios de aceite e matriz de cobertura de testes.
Pedro Iranildo	DevOps Engineer	Dockerfile multi-stage, compose.yaml, esteira CI/CD em DAG no Actions e infraestrutura de deploy.
Ricardo Severiano de Souza Filho	Product Owner	Priorização do backlog, story map, rastreabilidade dos requisitos acadêmicos e consolidação documental.

2. Links Oficiais para Avaliação Acadêmica

Recurso	Endereço URL	Finalidade de Avaliação
Repositório GitHub	github.com/Luizrocha0/Proj-EAD	Código-fonte auditado, branches e documentação técnica completa.
Pull Request de Entrega	github.com/Luizrocha0/Proj-EAD/pull/1	PR #1 aprovado e mesclado na branch `main` com status checks verdes.
Aplicação em Produção	rota-vital.onrender.com	Instância pública ativa na nuvem rodando sob perfil `infra`.
Health Check (Actuator)	rota-vital.onrender.com/actuator/health	Verificação de saúde com PostgreSQL ativo (`{"status":"UP"}`).
API REST de Estoque	rota-vital.onrender.com/api/estoque	Endpoint funcional com envelope padronizado (HTTP 200 OK).
Esteira CI/CD (Actions)	github.com/Luizrocha0/Proj-EAD/actions	Execução dos 6 gates automatizados em DAG sem contornos permissivos.

3. Síntese Técnica das Entregas (E1, E2 e E3)

📦 Bloco E1 — Containerização e Docker Compose

- **Dockerfile Multi-Stage sem Skip de Testes:** O estágio builder executa obrigatoriamente `./mvnw package -DskipITs -q`. Qualquer quebra em testes unitários aborta o build na origem. Runtime Alpine mínimo com usuário dedicado non-root (`rotavital:rotavital`, UID 10001).
- **Consumo de Sinais POSIX:** O entrypoint invoca formalmente `exec java $JAVA_OPTS -jar app.jar`, assumindo o PID 1 e respondendo a `SIGTERM` para graceful shutdown.
- **Orquestração Local (`compose.yaml`):** Serviços `api` e `db` comunicando via rede bridge isolada `rotavital-network`, persistência no volume nomeado `rotavital-postgres-data` e healthchecks atômicos com `pg_isready`.
- **Segurança:** Arquivo `.env.example` documentado e segredos blindados no `.gitignore` e `.dockerignore`.

💉 Bloco E2 — TDD e Persistência Relacional

- **Arquitetura Hexagonal Mínima:** A interface `EstoqueStorage` desacopla a regra de negócio do meio físico: `ListaEstoqueStorage` (perfil `u1` em memória pura) e `JdbcEstoqueStorage` (perfil `infra` em PostgreSQL 16 via Spring JDBC).
- **Proibição de Fallback Silencioso:** Sob o perfil `infra`, falhas de conexão geram erro explícito e status DOWN no Actuator, sem mascaramento de dados em memória.
- **Histórias H1, H2 e H3:** Cadastro completo de bolsas com DTO resiliente (H1); listagem com envelope `{"bolsas": [...], "tamanho": N}` e ordenação LIFO estrita (H2); e remoção cirúrgica da ocorrência mais recente sob IDs repetidos via subquery SQL atômica com índice composto (H3).
- **Persistência Pós-Reinício:** A suíte `BolsaPersistidaIT` comprova que os dados sobrevivem ao reinício total do contexto Spring.

🚀 Bloco E3 — CI/CD, Deploy no Render e Governança

- **Esteira CI/CD em DAG:** 6 gates independentes no GitHub Actions sem flags permissivas (`continue-on-error` proibido).
- **Deploy Hook Seguro:** Disparo no Render repassando o commit SHA aprovado (`ref=${{ github.sha }}`) após validação de todos os 5 gates anteriores na `main`.
- **Governança de Rollback em 3 Vias:** Rollback imediato por webhook com SHA anterior, reversão manual no dashboard do Render e reversão auditada via `git revert` com novo PR.
- **Proteção de Esquema (Flyway):** Estratégia Expand/Contract com `spring.flyway.clean-disabled=true` ativado em produção.
- **Branch Protection Ativa:** A branch `main` está formalmente protegida no GitHub (`"protected": true`) com 5 status checks obrigatórios estritos.

4. Bateria de Testes e Validação Empírica

Suíte / Nível de Teste	Tecnologia / Ferramenta	Quantidade	Tempo Médio	Resultado Verificado
Testes Unitários Java (Nível 1)	Maven Surefire / JUnit 5	**201 testes**	~22.0 s	✅ **201/201 Aprovados (0 falhas, 0 erros)**
Testes de Integração & Aceitação	Maven Failsafe + Testcontainers (PostgreSQL 16)	**39 testes**	~35.0 s	✅ **39/39 Aprovados (banco real em contêiner)**
Persistência Pós-Reinício	`BolsaPersistidaIT` (Context Restart)	**3 cenários**	~8.0 s	✅ **Aprovado (sobrevivência dos dados comprovada)**
Módulo AED em C (U1)	GCC + AddressSanitizer + UBSan	**54 asserções**	~0.05 s	✅ **54/54 Aprovados (zero leaks de memória)**
Benchmarks SO em Python (U1)	Python `unittest` + Validador Estatístico	**7 testes**	~0.01 s	✅ **7/7 Aprovados (série histórica de 40 medições intacta)**
Smoke Test Containerizado	Docker Compose + cURL (CRUD de 9 etapas)	**9 passos**	~15.0 s	✅ **Aprovado (Actuator UP + 200, 400 e 404 testados)**

5. Matriz das 12 Decisões Arquiteturais Justificadas

#	Decisão Escolhida	Alternativa Considerada	Justificativa Técnica Concreta
1	**Eclipse Temurin 21 JRE Alpine**	Ubuntu / Debian slim	Imagem oficial LTS enxuta (< 180MB), reduzindo vulnerabilidades e tempo de deploy.
2	**Build Multi-Stage sem `-DskipTests`**	Build monolítico / skip de testes	Garante que nenhum JAR com testes quebrados seja empacotado na imagem final.
3	**PostgreSQL 16 Oficial Alpine**	H2 em memória / SQLite	Paridade total de produção (12-Factor App) e compatibilidade nativa com Testcontainers.
4	**Spring Data JDBC puro (`JdbcTemplate`)**	Spring Data JPA / Hibernate	Controle absoluto do SQL para subqueries atômicas LIFO sem overhead de proxies ou cache.
5	**Migrações com Flyway**	`ddl-auto: update` / scripts manuais	Evolução declarativa, rastreável e auditável com validação automática de checksums.
6	**Volume Nomeado `rotavital-postgres-data`**	Bind mount no host	Persistência gerenciada pelo Docker, imune a permissões de arquivos no host Windows/Linux.
7	**Rede Bridge Isolada `rotavital-network`**	Rede host / rede default	API e banco comunicam-se por DNS interno sem expor portas do PostgreSQL para a internet.
8	**Pirâmide em 3 Níveis (Surefire vs Failsafe)**	Execução mista única	Nível 1 roda em segundos; Níveis 2 e 3 validam banco real via Testcontainers com fail-fast.
9	**GitHub Flow com Branch de Trabalho**	Push direto na `main`	Commits atômicos, PR auditável, execução de gates e histórico de entregas preservado.
10	**Esteira CI/CD em DAG com 6 Gates**	Pipeline monolítico linear	Paralelismo inicial entre Python, C e Java unitário, convergindo para Testcontainers e Deploy.
11	**Deploy Hook no Render com `ref=${{ github.sha }}`**	Auto-deploy do Render	Publica exclusivamente o commit SHA aprovado por todos os 5 gates do pipeline.
12	**Segredos em Cofres (GitHub Secrets e `.env`)**	Chaves hardcoded no código	Blindagem de credenciais sensíveis conforme o 12-Factor App e LGPD.

🤖 Declaração Transparente de Uso de Inteligência Artificial

Em estrita conformidade com as diretrizes acadêmicas de integridade e ética: o desenvolvimento contou com a assistência do **Google Antigravity / Gemini 2.0** como copiloto de programação para geração de scaffolding de testes unitários, automação de scripts de CI/CD e suporte à formatação documental. **Toda a concepção arquitetural** (arquitetura hexagonal mínima, modelagem relacional com chave técnica autoincrementável `id_seq`, subqueries SQL atômicas para LIFO, política de perfis sem fallback silencioso e governança de rollback) foi formulada, revisada, validada e auditada pela equipe humana, garantindo domínio conceitual pleno para a apresentação em sala de aula.
